

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	PRUEBAS DE SOFTWARE				
TÍTULO DE LA PRÁCTICA:	Pruebas Unitarias con JUnit				
NÚMERO DE PRÁCTICA:	04	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	X	MÁXIMO DE ESTUDIANTES	4	
FECHA INICIO:	11/05/2026	FECHA FIN:	15/05/2026	DURACIÓN:	100 min
RECURSOS A UTILIZAR: - Una computadora personal - Lenguaje de programación Java - Visual Studio Code + extensión Java - Framework de pruebas unitarias Junit 5.0 - Maven 3.9 / JaCoCo 0.8 para análisis de cobertura de código					
DOCENTE(s): Prof. Robert Arisaca / Prof. Diego Iquira / Prof. Lino Pinto					
OBJETIVOS/TEMAS Y COMPETENCIAS					
OBJETIVOS: - El estudiante comprenderá la importancia de diseñar pruebas unitarias de componentes de software basados en Java - El estudiante ejecutará pruebas unitarias basadas en el framework JUnit 5.0 y realizará un análisis de cobertura con JaCoCo					
TEMAS: - Pruebas Unitarias: definición y propósito - Historia y evolución de JUnit - Apache Maven y gestión de dependencias - Anotaciones y ciclos de vida en JUnit 5 - Pruebas parametrizadas / Métricas de cobertura (JaCoCo)					

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 2

COMPETENCIAS	C.m Construye responsablemente soluciones siguiendo un proceso adecuado llevando a cabo las pruebas ajustadas a los recursos disponibles del cliente.
---------------------	---

CONTENIDO DE LA GUÍA	
I. MARCO CONCEPTUAL 1.1 Historia y Evolución de JUnit JUnit fue creado en 1997 por Kent Beck y Erich Gamma, motivado por la necesidad de automatizar pruebas en Java. Beck fue influenciado por el trabajo de Smalltalk en SUnit, adaptando el concepto al lenguaje Java. Esta herramienta revolucionó la industria del software al facilitar la implementación de Test-Driven Development (TDD). Evolución de JUnit: • JUnit 1.x (1997-2000): Primera versión, basada en herencia y xUnit pattern • JUnit 3.x (2000-2006): Popularización, uso de herencia de TestCase y métodos setUp/tearDown • JUnit 4.x (2006-2017): Introducción de anotaciones (@Test, @Before, @After), mayor flexibilidad • JUnit 5.0 (2017-2022): Arquitectura modular, lambda support, extensiones, JUnit Platform • JUnit 5.10+ (2023-presente): Mejoras en parametrización, mejor integración con herramientas modernas 1.2 Arquitectura de JUnit 5 JUnit 5 fue rediseñado con una arquitectura modular que separa las concerns de diferentes actores en el ecosistema de testing: • JUnit Platform: Fundación para ejecutar tests en la JVM. Proporciona TestEngine y APIs de extensión. • JUnit Jupiter: Nuevo modelo de programación. Incluye annotations, assertions y extensiones. • JUnit Vintage: Proporciona compatibilidad retroactiva con JUnit 3.x y 4.x a través de TestEngine. 1.3 Apache Maven y Gestión de Dependencias Apache Maven es una herramienta de construcción y gestión de proyectos Java que utiliza un modelo declarativo basado en un archivo de configuración (pom.xml). Maven soluciona problemas comunes en proyectos Java como la gestión de dependencias, compilación, testing y empaquetamiento. • Gestión centralizada de dependencias con transitividad automática • Ciclo de vida estándar de construcción (clean, compile, test, package, install, deploy) • Integración con repositorios remotos (Maven Central) • Ejecución automática de pruebas unitarias • Generación de reportes y documentación 1.4 Configuración de pom.xml para JUnit 5 El archivo pom.xml (Project Object Model) es el corazón de un proyecto Maven. Define metadatos del proyecto, dependencias, plugins y configuraciones de construcción. A continuación, se presenta una configuración parcial para JUnit 5: <pre><?xml version="1.0" encoding="UTF-8"?> <project xmlns="http://maven.apache.org/POM/4.0.0"</pre>	

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>

<!-- Información del Proyecto -->
<groupId>com.lab04</groupId>
<artifactId>pruebas-unitarias-junit</artifactId>
<version>1.0.0</version>
<name>Pruebas Unitarias con JUnit 5</name>
<description>
    Guía de laboratorio para pruebas unitarias en Java 25 con JUnit 5
</description>
</project>
```

Para usar este pom.xml: 1) Crear directorio de proyecto, 2) Colocar pom.xml en raíz, 3) Ejecutar "mvn clean test" en terminal.

1.5 Estructura de Directorios de un Proyecto Maven

```
proyecto/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/lab04/
│   │   │       ├── Calculadora.java
│   │   │       └── CuentaBancaria.java
│   │   └── resources/
│   └── test/
│       ├── java/
│       │   └── com/lab04/
│       │       ├── CalculadoraTest.java
│       │       └── CuentaBancariaTest.java
│       └── resources/
└── target/
    ├── classes/
    └── test-classes/
```

1.6 Anotaciones Principales en JUnit 5

- **@Test:** Marca un método como prueba unitaria ejecutable
- **@DisplayName:** Proporciona un nombre personalizado para la prueba (con espacios y caracteres especiales)
- **@BeforeEach:** Se ejecuta antes de CADA prueba
- **@AfterEach:** Se ejecuta después de CADA prueba
- **@BeforeAll:** Se ejecuta una sola vez antes de TODAS las pruebas (debe ser static)
- **@AfterAll:** Se ejecuta una sola vez después de TODAS las pruebas (debe ser static)
- **@ParameterizedTest:** Marca una prueba para ejecutarse múltiples veces con diferentes parámetros
- **@ValueSource:** Proporciona una fuente de valores para pruebas parametrizadas

- **@CsvSource:** Proporciona datos en formato CSV para pruebas parametrizadas
- **@Disabled:** Deshabilita temporalmente una prueba
- **@Nested:** Anida una clase de prueba dentro de otra para agrupar pruebas relacionadas
- **@Tag:** Etiqueta pruebas para filtrado y categorización

1.6 Assertions en JUnit 5

- **assertEquals(expected, actual):** Verifica que dos valores sean iguales
- **assertTrue(condition):** Verifica que una condición sea verdadera
- **assertFalse(condition):** Verifica que una condición sea falsa
- **assertNull(object):** Verifica que un objeto sea nulo
- **assertNotNull(object):** Verifica que un objeto NO sea nulo
- **assertThrows(ExceptionType, executable):** Verifica que se lance una excepción específica
- **assertDoesNotThrow(executable):** Verifica que NO se lance ninguna excepción
- **assertAll(...):** Ejecuta múltiples assertions, reportando todas las fallas
- **assertArrayEquals(expected, actual):** Verifica que dos arrays sean iguales
- **assertIterableEquals(expected, actual):** Verifica que dos Iterables sean iguales

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

EJERCICIO 1: CALCULADORA SIMPLE

1.1 Descripción del Problema

Se requiere crear una clase Calculadora que realice operaciones matemáticas básicas (suma, resta, multiplicación, división). Cada operación debe ser probada a nivel de pruebas unitarias para lograr una cobertura del 100%

1.2 Requisitos Funcionales

- Suma de dos enteros
- Resta de dos enteros
- Multiplicación de dos enteros
- División de dos enteros (resultado double)
- Manejo de división entre cero con excepción

1.3 Código de la Clase a Probar

Archivo: src/main/java/pe/com/lab04/Calculadora.java

1.4 Código de la Clase que contiene las pruebas unitarias

Archivo: src/main/java/pe/com/lab04/CalculadoraTest.java

1.5 Ejecución y Resultados

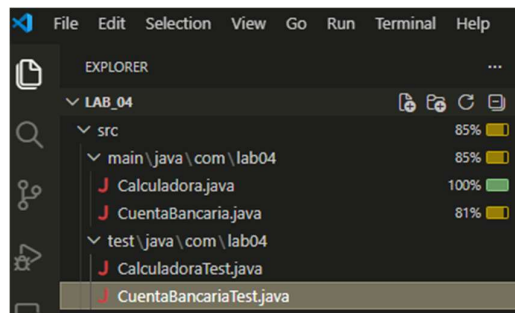
Para ejecutar las pruebas:

- `> mvn clean test`

- > mvn test -Dtest=CalculadoraTest

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.lab04.CalculadoraTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.112 s - in com.lab04.CalculadoraTest
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
```

- Desde VSCode: Clic derecho en CalculadoraTest.java > Run Test with Coverage



1.6 Puntos Clave del Ejercicio

- @Test marca métodos como pruebas
- @DisplayName proporciona descripciones legibles
- @BeforeEach inicializa el objeto antes de cada prueba
- assertEquals verifica igualdad
- assertThrows verifica que se lance excepción
- Patrón AAA: Arrange-Act-Assert
- Cobertura de todos los casos

EJERCICIO 2: GESTOR DE CUENTA BANCARIA

2.1 Descripción del Problema

Se requiere crear una clase CuentaBancaria que gestione transacciones bancarias (depósitos, retiros) con validaciones y manejo de estado. Este ejercicio introduce pruebas parametrizadas y ciclo de vida más complejo.

2.2 Requisitos Funcionales

- Mantener número de cuenta, saldo inicial y titular
- Permitir depósitos con validación de monto positivo
- Permitir retiros con validación de saldo suficiente
- Registrar historial de transacciones
- Lanzar excepciones para operaciones inválidas
- Consultar saldo actual

2.3 Código de la Clase a Probar

Archivo: src/main/java/pe/com/lab04/CuentaBancaria.java

1.4 Código de la Clase que contiene las pruebas unitarias

Archivo: src/main/java/pe/com/lab04/CuentaBancariaTest.java

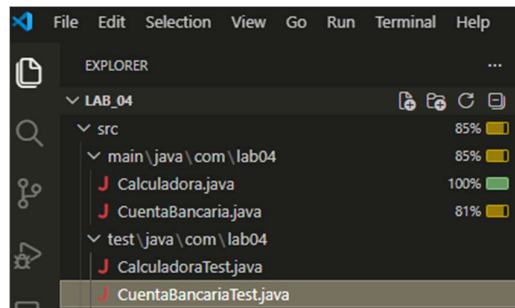
1.5 Ejecución y Resultados

Para ejecutar las pruebas:

- > mvn clean test
- > mvn test -Dtest=CuentaBancariaTest

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.lab04.CuentaBancariaTest
Prueba finalizada. Saldo final: 1000.0
Prueba finalizada. Saldo final: 900.0
Prueba finalizada. Saldo final: 500.0
Prueba finalizada. Saldo final: 0.0
Prueba finalizada. Saldo final: 1000.0
Prueba finalizada. Saldo final: 1000.0
Prueba finalizada. Saldo final: 1500.0
Prueba finalizada. Saldo final: 1050.0
Prueba finalizada. Saldo final: 1100.0
Prueba finalizada. Saldo final: 1250.5
Prueba finalizada. Saldo final: 2000.0
Prueba finalizada. Saldo final: 3500.0
Prueba finalizada. Saldo final: 1000.0
Prueba finalizada. Saldo final: 1000.0
Prueba finalizada. Saldo final: 700.0
Prueba finalizada. Saldo final: 1000.0
[INFO] Tests run: 16, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.461 s - in com.lab04.CuentaBancariaTest
[INFO] Results:
[INFO]
[INFO] Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
```

- Desde VSCode: Clic derecho en CuentaBancariaTest.java > Run Test with Coverage



1.6 Puntos Clave del Ejercicio

- @BeforeEach y @AfterEach para setup/teardown
- @ParameterizedTest para pruebas con múltiples valores
- @ValueSource para valores simples
- @CsvSource para múltiples parámetros y resultados
- Validaciones en constructores

- Mantenimiento de estado (saldo, transacciones)
- Assertions con mensajes descriptivos
- Uso de LocalDateTime para timestamps

III. EJERCICIOS/PROBLEMAS PROPUESTOS

EJERCICIO PROPUESTO 1: GESTOR DE INVENTARIO

1.1 Descripción General

Desarrolle un programa para la gestión de inventario que permita controlar el stock de productos. El programa debe mantener información de productos y registrar movimientos (entradas y salidas). Este ejercicio enfatiza validaciones y casos límite.

1.2 Requisitos Funcionales

- Clase Producto con atributos: código, nombre, precio, cantidad
- Validar que el código no sea vacío
- Validar que la cantidad nunca sea negativa
- Validar que el precio sea positivo
- Método agregarStock(cantidad): incrementa el stock
- Método extraerStock(cantidad): decrementa el stock
- Método consultarStock(): retorna cantidad disponible
- Método obtenerValorTotal(): retorna precio * cantidad
- Lanzar excepciones para operaciones inválidas
- Registrar fecha y hora de cada movimiento

1.3 Requisitos de Pruebas

- Mínimo 12 casos de prueba
- Cobertura de código al 100%
- Cobertura de casos excepcionales (valores negativos, nulos)
- Al menos una prueba parametrizada
- Usar @BeforeEach para inicialización
- Usar @DisplayName para descripciones legibles
- Validar mensajes de error en excepciones

1.4 Sugerencias de Estructura de Clases

- Clase Producto:
 - Atributos: código (String), nombre (String), precio (double), cantidad (int)
 - Métodos: getters, agregarStock(), extraerStock(), consultarStock()

- Clase Movimiento:
 - Atributos: tipo (ENTRADA/SALIDA), cantidad, fecha (LocalDateTime)

EJERCICIO PROPUESTO 2: CARRITO DE COMPRAS

2.1 Descripción General

Implemente un programa de carrito de compras para una tienda en línea. Este ejercicio integra múltiples clases, operaciones complejas y requiere uso de Mockito para simular servicios externos como cálculo de impuestos y descuentos.

3.2 Requisitos Funcionales

- Clase Producto: id, nombre, precio, disponibilidad
- Clase CarritoCompra: agregar producto, remover producto, vaciar
- Interfaz ServicioPrecio: calcularDescuento(), calcularImpuesto()
- Operación: calcularTotal() con descuentos e impuestos
- Operación: obtenerResumenCompra()
- Validar que no se agreguen productos indisponibles
- Validar que no se agregue cantidad negativa
- Permitir actualizar cantidad de producto existente
- Detectar productos duplicados en carrito
- Historial de operaciones del carrito

3.3 Requisitos de Pruebas

- Mínimo 18 casos de prueba
- Pruebas sin mocks: operaciones básicas de carrito
- Pruebas con Mockito: ServicioPrecio simulado
- Pruebas parametrizadas para diferentes montos
- Validar cálculo correcto de total con impuesto
- Validar cálculo correcto de total con descuento
- Validar carrito vacío (total = 0)
- Validar producto duplicado
- Verificar llamadas a ServicioPrecio
- Casos límite: carrito con 1 producto, con 100 productos
- Usar @Nested para agrupar pruebas relacionadas
- Cobertura mínima 85%

3.4 Estructura Sugerida

- Producto:
 - Atributos: id, nombre, precio (double), disponible (boolean)

- CarritoCompra:
 - Atributos: items (List<ItemCarrito>), servicioPrecio
 - Métodos: agregar, remover, calcularTotal, obtenerResumen
- ServicioPrecio (Interfaz):
 - Métodos: calcularDescuento(double), calcularImpuesto(double)

I. CUESTIONARIO

Pregunta 1: Principios FIRST y Pruebas Independientes

¿Por qué es crítico el principio "Independent" (Independencia) en las pruebas unitarias? Explique las consecuencias de tener pruebas dependientes entre sí. ¿Cómo ayuda Mockito y la inyección de dependencias a lograr independencia total?

Pregunta 2: Anotaciones de Ciclo de Vida en JUnit 5

¿Cuál es el propósito de las anotaciones de ciclo de vida (@BeforeEach, @AfterEach, @BeforeAll, @AfterAll)? Proporcione casos de uso específicos para cada una, explique cuándo usar cada una y por qué son necesarias. Incluya al menos un anti-patrón (mal uso).

Pregunta 3: Mejora de Calidad con Mockito y Inyección de Dependencias

¿Cómo mejora Mockito la calidad de las pruebas unitarias en comparación con pruebas de integración? Discuta las diferencias entre mocks, stubs y spies. ¿Cuándo es apropiado usar cada uno? Proporcione ejemplos del mundo real (e.g., APIs externas, bases de datos, servicios de pago).

Pregunta 4: Testing y Deuda Técnica

¿Cuál es la relación entre las pruebas unitarias exhaustivas y la reducción de deuda técnica? Analice cómo las pruebas facilitan refactorización segura y prevención de regresiones. Investigue proyectos Open Source exitosos y cómo manejan testing.

II. REFERENCIAS Y BIBLIOGRAFÍA RECOMENDADAS:

[1] Beck, K., & Gamma, E. (1997). *"Test Infected: Programmers Love Writing Tests"*. Java Report, 7(9), 37-50. — Artículo seminal que introduce JUnit y revoluciona el testing en Java. Lectura fundamental.

[2] Fowler, M. (2018). *"Refactoring: Improving the Design of Existing Code"* (2nd ed.). Addison-Wesley Professional. — Clásico de refactorización. Capítulos 3-4 sobre testing y cambios seguros.

[3] Roman, S., Liguori, S., & Stanchev, P. (2022). *"JUnit in Action"* (3rd ed.). Manning Publications. — Guía comprehensiva de JUnit 5 con ejercicios prácticos y mejores prácticas.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 10

[4] Osherove, R. (2013). *"The Art of Unit Testing: with Examples in Java" (2nd ed.)*. Manning Publications. — Mejores prácticas para escribir pruebas efectivas, mantenibles y valiosas.

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:

*Ejercicios propuestos /
Cuestionario*

INSTRUMENTOS:

Rúbrica

CRITERIOS DE EVALUACIÓN

Criterio	Excelente (100%)	Bueno (80%)	Suficiente (55%)	Insuficiente (30%)	No presenta (0%)
Claridad y completitud de la explicación de los ejercicios propuestos. (30 %)	Todos los ejercicios propuestos han sido explicados de forma muy clara y con la profundidad adecuada.	Los ejercicios propuestos han sido resueltos y explicados, pero de manera muy superficial o general.	Los ejercicios propuestos han sido resueltos, sin embargo, algunos de ellos no han sido explicados.	Los ejercicios han sido resueltos parcialmente y no tienen explicaciones de lo realizado.	No presentan los ejercicios resueltos, o la versión final del informe omite por completo esta sección.
Ejercicios Propuestos (Código fuente) (20 %)	Todos los ejercicios propuestos se ejecutan correctamente y devuelven los resultados esperados.	Los ejercicios propuestos han sido implementados pero con los resultados esperados parciales.	Por lo menos el 2/3 de los ejercicios propuestos han sido implementados con resultados	Por lo menos el 1/3 de los ejercicios propuestos han sido implementados con resultados	No se han implementado los ejercicios propuestos.
Cuestionario (30%)	Todas las respuestas son completas, correctas y proporcionan explicaciones claras, incluyen citas.	La mayoría de las respuestas son correctas, pero algunas carecen de la profundidad esperada, pocas citas.	Algunas respuestas son correctas, pero incompletas o con falta de claridad en las explicaciones, citas no relevantes.	La mayoría de las respuestas son escuetas y/o incompletas, no hay evidencia de haber buscado información que las	No presenta el cuestionario.



				respalden, sin citas.	
Presentación del Informe (20%)	El informe está bien estructurado, con excelente redacción, sin errores	El informe está bien estructurado, con algunos errores menores de redacción	El informe presenta problemas en la organización o redacción, con varios errores	El informe está desorganizado, con múltiples errores de redacción y formato	No presenta el informe.